
API Compatibility Is Not Gradient Compatibility: Projection-Only LoRA Fails on Mamba-130m for Classification Tasks

Anonymous Authors¹

Abstract

Low-rank adaptation (LoRA) is widely applied to transformer models by targeting `nn.Linear` projection layers — a recipe that transfers, practitioners assume, to any architecture with equivalent layer types. We test this assumption on Mamba-130m, a pure state space model (SSM) whose projection layers are `nn.Linear` and whose community documentation reports 90–92% SST-2 accuracy with projection-only LoRA. Applying LoRA to `in_proj`, `out_proj`, and `x_proj` (rank 8, 1.14% trainable parameters), we find that SST-2 accuracy remains at 0.5092 across all three training epochs — statistically indistinguishable from the zero-shot baseline of 0.4908 — while MNLI accuracy degrades from 0.3463 to 0.3234, both results well below the conservative gate criterion of 0.70. The failure is not a configuration error: LoRA installs correctly and adapter weights receive gradient updates, but those updates carry no task-discriminative information. We attribute this pattern to the Mamba selective scan kernel, a custom CUDA operation that we interpret as acting as a gradient barrier between the classification head and the projection-layer adapters; gradient magnitudes were not directly logged, so this remains a medium-confidence mechanistic interpretation. These findings establish that API compatibility with `nn.Linear` is not a sufficient condition for LoRA to produce learning on SSM architectures, and that Mamba parameter-efficient fine-tuning requires gradient-path-aware methods that operate inside or upstream of the SSM scan.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

1. Introduction

We applied LoRA to Mamba-130m exactly as we would to any transformer — correct target modules, correct parameter counts, 1.14% trainable weights — and watched SST-2 accuracy sit at 50.9% for three consecutive training epochs, indistinguishable from the zero-shot baseline of 49.1%. This is not slow convergence. This is zero learning.

The result is surprising precisely because everything visible looked correct. The PEFT library installed the adapters without complaint. The model’s state dictionary contained LoRA weight matrices at every targeted projection layer. The training loop ran, loss values were logged, and gradients were computed. And yet, after three full epochs over 4,000 training samples — a protocol sufficient for LoRA to converge on SST-2 with transformers — the model remained locked at majority-class prediction accuracy, as if fine-tuning had never occurred.

1.1. The Surface Problem

Parameter-efficient fine-tuning, and LoRA in particular (Hu et al., 2022), has become the dominant strategy for adapting large pretrained language models to downstream tasks. By freezing the base model and training only low-rank decompositions of selected weight matrices, LoRA achieves near-full-fine-tuning accuracy at a fraction of the trainable parameter cost. The method’s appeal is partly its apparent architecture-agnosticism: any layer implemented as `nn.Linear` is a candidate for LoRA adaptation.

Mamba (Gu & Dao, 2023) is the leading transformer-alternative for long-context language modeling. Its selective state space mechanism achieves linear-time training and constant-memory inference, making it a natural target for deployment scenarios where transformer attention is prohibitively expensive. Mamba-130m is publicly available on the HuggingFace Hub, its projection layers — `in_proj`, `out_proj`, and `x_proj` — are all `nn.Linear`, and community implementations (alxndrTL/mamba-peft) report SST-2 accuracy of 90–92% with LoRA rank 8 on these exact layers.

The natural inference is that LoRA on Mamba works. Practitioners building Mamba-based NLP pipelines would rea-

sonably apply LoRA to these projection layers and expect learning to occur.

1.2. The Deeper Problem and the Gap

The assumption that LoRA on `nn.Linear` produces learning is valid for transformers because it rests on an implicit guarantee: the gradient path from the task loss to the LoRA weight matrices is unobstructed. In transformers, the attention mechanism is implemented as fully differentiable operations under standard PyTorch autograd. Any `nn.Linear` target in the attention stack receives well-defined gradients when trained on any loss function.

In Mamba, this guarantee does not hold. The architecture’s core computation — the selective state space scan $h_t = \bar{A} \cdot h_{t-1} + \bar{B} \cdot x_t$ — is implemented as a custom CUDA parallel associative scan kernel. This kernel is not a standard autograd-differentiable operation in the same sense as `nn.Linear` or `softmax`. When a randomly-initialized classification head is placed atop a pretrained causal Mamba model and trained with cross-entropy loss, the classification gradient must flow backward through this SSM scan kernel to reach the LoRA weight matrices in the projection layers.

Our experiments demonstrate that this gradient path is ineffective for classification. Three training epochs produce exactly zero improvement over zero-shot accuracy on SST-2 (50.9% vs. 49.1% zero-shot, a net gain of 1.84 percentage points — within noise). MNLI accuracy actively degrades from its zero-shot level of 34.6% to 32.3% by the second epoch. The loss oscillates between 0.65 and 0.73 across training batches without monotonic decrease — the signature of incoherent gradient updates rather than directed learning.

No prior work, to our knowledge, provides a controlled, reproducible characterization of projection-only LoRA failure on pure Mamba SSMs for classification tasks. The community reference (alxndrTL/mamba-peft, 2024) reports 90–92% SST-2 without fully documenting the training recipe. Practitioners attempting to replicate this result with the base `mamba-130m-hf` checkpoint, a standard classification head, and a clean training loop will observe what we observed: no learning. Without a clear failure characterization, the field will repeatedly rediscover this failure independently.

1.3. Key Insight

We attribute the failure to the Mamba selective state-space scan, which we interpret as acting as an effective gradient barrier between classification loss and LoRA weight matrices — a property invisible from the PEFT API but immediately apparent from per-epoch accuracy tracking.

The SSM scan was designed and optimized for next-token

prediction, the task on which Mamba-130m was pretrained. For a randomly-initialized classification head placed on the final hidden state, the gradient is novel and apparently untranslatable through the same path: it exists numerically but produces near-zero directed updates to the LoRA weight matrices, leaving the model locked at its pretraining prediction regime.

This is a conceptual distinction the PEFT community needs: API compatibility — the ability to install LoRA adapters with correct parameter counts — is a necessary but not sufficient condition for fine-tuning to produce learning. Gradient compatibility, verified empirically through per-epoch task accuracy, is the authoritative criterion.

1.4. Contributions

Our investigation reveals three contributions:

First, we provide direct empirical evidence that projection-only LoRA (targeting `in_proj`, `out_proj`, `x_proj` with rank 8) does not produce meaningful classification task learning on Mamba-130m. The evidence is precise: SST-2 accuracy of 0.5092 across three identical consecutive training epochs (zero-shot baseline: 0.4908), and MNLI accuracy degrading from 0.3463 zero-shot to 0.3234 by the second epoch.

Second, we document verified zero-shot GLUE baselines for Mamba-130m: SST-2 = 0.4908, MNLI = 0.3463, QNLI = 0.5056, QQP = 0.0000. These baselines, confirmed across two independent evaluations, provide a reproducible reference for future Mamba adaptation experiments.

Third, we characterize the SSM scan gradient barrier as the most consistent mechanistic interpretation of the failure, identify a two-signal diagnostic signature for practitioners (loss oscillation without monotonic decrease, combined with per-epoch accuracy plateau), and specify the design constraint this places on future Mamba PEFT methods.

1.5. Paper Organization

Section 2 reviews Mamba’s architectural properties, existing PEFT methods, and the MambaPEFT literature. Section 3 describes our experimental design. Section 4 presents the experimental setup. Section 5 reports results. Section 6 interprets results and discusses limitations. Section 7 concludes with future directions.

2. Related Work

Our work sits at the intersection of three bodies of literature: parameter-efficient fine-tuning methods, Mamba and state-space sequence models, and the growing body of negative results in machine learning.

2.1. LoRA and Parameter-Efficient Fine-Tuning

Low-Rank Adaptation (LoRA) (Hu et al., 2022) introduced the standard approach of decomposing weight updates into low-rank factors: for a frozen weight matrix W , the update is parameterized as $\Delta W = BA$ where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ with rank $r \ll \min(d, k)$. The method was validated on GPT-2, GPT-3, and RoBERTa across a range of NLP tasks. The original work explicitly targets attention matrices (W_q, W_k, W_v, W_o) — all `nn.Linear` layers in transformer architectures where gradient flow from task loss to LoRA weights is unconditionally guaranteed.

Subsequent PEFT methods extended LoRA’s ideas. AdaLoRA (Zhang et al., 2023) introduces adaptive rank allocation across layers. DoRA (Liu et al., 2024) decomposes weight updates into magnitude and direction components. IA³ (Liu et al., 2022) applies multiplicative rescaling rather than additive low-rank updates. Each of these methods was designed for and validated on transformer architectures, where the critical assumption — that gradient flows cleanly to any targeted `nn.Linear` layer — is satisfied by construction.

The assumption is implicit, not stated. Because transformers always satisfy it, the PEFT literature has not needed to treat gradient path compatibility as a design criterion. This is the gap our work fills.

2.2. Mamba and State-Space Sequence Models

Structured state space models have evolved from S4 (Gu et al., 2022), which demonstrated that SSMs could match recurrent and convolutional architectures on long-range dependency tasks. Mamba (Gu & Dao, 2023) introduced *selectivity*: rather than fixed state transition matrices, Mamba’s selective SSM (S6) computes A , B , and C as functions of the input. This selectivity is implemented via a custom CUDA parallel associative scan kernel. The Mamba paper characterizes this mechanism for its modeling capabilities; the gradient path properties of this kernel for classification tasks are not addressed.

Mamba-2 (Dao & Gu, 2024) reformulates the state space computation via the Structured State Space Duality (SSD) framework, unifying SSMs and attention under a common theoretical lens. The broader SSM family includes RWKV (Peng et al., 2023) and RetNet (Sun et al., 2023). None of these architectures have been systematically characterized for PEFT gradient compatibility.

2.3. Mamba PEFT: The Prior Claim We Contrast Against

The primary public reference for LoRA on Mamba is `alxndrTL/mamba-peft` (alxndrTL, 2024), a commu-

nity repository demonstrating SST-2 accuracy of approximately 90–92% using LoRA rank 8 applied to `in_proj`, `out_proj`, and `x_proj` on Mamba-130m. This is the claim that motivated our systematic investigation.

We do not claim that the mamba-peft result is wrong. We note that it is insufficiently documented to reproduce: the checkpoint used, the classification head design, the prompt format, and the complete learning rate schedule are not fully specified. The 40 percentage-point gap between our result and theirs is an open reproducibility problem. Our result may reflect the failure mode one encounters with the base pretrained checkpoint and a standard training recipe.

2.4. Prefix Tuning and Bypass PEFT Strategies

Li & Liang (2021) introduced prefix tuning: rather than modifying model weights, the method prepends learned continuous token embeddings to the input. The key property for our discussion is that prefix tuning does not require gradients to flow through the model’s internal computations. For Mamba, prefix tuning would steer the SSM’s hidden state trajectory through learned input perturbations without requiring classification gradients to propagate backward through the SSM scan kernel — making it a theoretically motivated bypass strategy.

2.5. Negative Results in Machine Learning

The machine learning community has increasingly recognized the value of documented negative results (Sinha et al., 2021). The principled documentation of a failure mode — with a mechanistic interpretation, a diagnostic signature, and a forward direction — is a scientific contribution distinct from performance improvements.

To our knowledge, no prior work provides a controlled, reproducible characterization of where and how projection-only LoRA fails on pure Mamba SSMs for classification tasks.

3. Methodology

We designed experiment h-e1 as a **MUST_WORK existence check** — a protocol that gives projection-only LoRA every possible advantage and treats failure as an unambiguous signal rather than a tuning problem. If LoRA is going to work on Mamba for classification at all, it should work under the conditions we construct.

The key diagnostic insight: we measured **per-epoch accuracy alongside loss** at every training epoch. Loss curves can look plausible — oscillating, non-zero, apparently “doing something” — while accuracy reveals complete learning failure. Three consecutive identical accuracy values across epochs cannot be rationalized as slow convergence or un-

Figure 1. Mamba gradient path diagram. The selective SSM scan kernel sits between the classification head and the LoRA weight matrices in `in_proj/out_proj/x_proj` across all 24 layers. The dashed arrow from classification head through SSM scan to LoRA matrices indicates the non-discriminative gradient path.

Figure 2. Per-epoch accuracy curves. Left: SST-2 accuracy across epochs (flat line at 0.5092; zero-shot reference at 0.4908; gate criterion at 0.70). Right: MNLI accuracy (0.3463 zero-shot $\rightarrow$ 0.3326 epoch 1 $\rightarrow$ 0.3234 epoch 2).

lucky initialization.

3.1. Mamba Architecture: Selective State Space Scan and Gradient Flow

Mamba-130m (`state-spaces/mamba-130m-hf`) is a 24-layer causal language model with hidden dimension $d_{\text{model}} = 768$, inner dimension $d_{\text{inner}} = 1536$ (expansion factor 2), state dimension $d_{\text{state}} = 16$, and discrete-time rank $dt_rank = 48$. Each layer contains a MambaBlock with the following `nn.Linear` projection layers: `in_proj` (768 $\rightarrow$ 3072), `out_proj` (1536 $\rightarrow$ 768), `x_proj` (1536 $\rightarrow$ 112), and `dt_proj` (48 $\rightarrow$ 1536).

The MambaBlock also contains a `conv1d` layer and, critically, the **selective SSM scan**: the computation $h_t = \bar{A}(\Delta, A) \cdot h_{t-1} + \bar{B}(\Delta, B) \cdot x_t$, $y_t = C \cdot h_t$, implemented as a custom CUDA parallel associative scan (`selective_scan_cuda`).

The gradient path for a classification loss placed on the final hidden state runs: classification head $\rightarrow$ final hidden state $\rightarrow$ (through scan and projection layers of all 24 layers) $\rightarrow$ LoRA weight matrices. The SSM scan sits *between* the classification head and the LoRA targets in every layer. For the model’s pretraining task (next-token prediction), this backward path through the scan is well-exercised. For a randomly-initialized classification head computing sequence-level predictions, the backward path is novel — and as our experiments demonstrate, apparently ineffective at communicating task-discriminative signal.

3.2. LoRA Application to Mamba-130m

We applied LoRA using the HuggingFace PEFT library (v0.9+) with the configuration in Table 1, matching the community reference (alxndrTL, 2024).

The classification head is a single `nn.Linear(768, num_labels)` layer applied to the last-token hidden state from the final MambaBlock. The head is randomly initialized.

Table 1. LoRA configuration for Mamba-130m (Condition A).

Hyperparameter	Value	Rationale
Target modules	<code>in_proj, out_proj, x_proj</code>	Community reference
Rank (r)	8	Standard for classification
Alpha (α)	16	$2r$; standard scaling
Dropout	0.05	Standard regularization
Trainable parameters	1,484,288 (1.14%)	Confirmed by PEFT

Table 2. Training configuration.

Hyperparameter	Value
Optimizer	AdamW
Learning rate	3×10^{-4}
Weight decay	0.01
Batch size	32
Epochs	3
LR schedule	Linear warmup (6%) + linear decay
Seed	42
Training samples	4,000 (subsampled)
Max sequence length	128 tokens
Tokenizer	GPT-NeoX-20B (50k vocab)

3.3. GLUE Fine-Tuning Setup

We fine-tune separately on SST-2 and MNLI. **MUST WORK gate criterion:** SST-2 accuracy > 0.70 after three epochs. The conservatism is deliberate: the community reference reports 90–92%, but a 70% gate gives LoRA a 20-percentage-point margin for training setup differences.

Why 3 epochs, 4,000 samples: An existence check detects whether a method can learn at all, not its ceiling performance. Three epochs of 4,000 samples is sufficient for LoRA to converge on SST-2 with transformer architectures — success, if achievable, would be visible.

3.4. Diagnostic Methodology

Our core design choice was **per-epoch accuracy measurement**, not just loss monitoring. Loss curves have multiple patterns consistent with either learning or non-learning. Accuracy is the authoritative criterion: three consecutive identical values across epochs eliminate all alternative explanations (slow convergence, lucky initialization plateau, seed sensitivity, evaluation bug).

We evaluate accuracy on the full validation split at the end of each epoch: SST-2 validation (872 examples), MNLI matched validation (9,815 examples). Before any fine-tuning, we evaluate the base Mamba-130m model on GLUE tasks using `lm-evaluation-harness` (Gao et al., 2023) with multiple-choice prompting to establish the

zero-shot reference.

Implementation correctness verification: Before accepting any result, we verify that (1) LoRA keys exist in the model’s state dictionary at expected module paths, (2) LoRA weight matrices are non-zero after training, and (3) the trainable parameter count matches the expected value. All three checks pass in our experiments.

4. Experimental Setup

This section describes the experimental design for h-e1, our MUST_WORK existence check for projection-only LoRA on Mamba-130m.

4.1. Research Questions

RQ1: Does standard projection-only LoRA produce any measurable classification learning on Mamba-130m? If the SSM scan gradient barrier hypothesis is correct, LoRA adapters should receive negligible task-discriminative gradient and produce no meaningful accuracy improvement over zero-shot.

RQ2: Is the failure mode, if present, consistent across multiple classification tasks? A task-specific failure could have a task-specific explanation; a cross-task failure with the same signature points to an architectural cause.

RQ3: Is the failure detectable from training-time diagnostics without waiting for epoch-end accuracy? The loss oscillation pattern provides a practitioner-relevant early-stopping signal.

4.2. Datasets

Table 3. GLUE tasks evaluated.

Dataset	Task	Train (used)	Val	Classes
SST-2	Binary sentiment	4,000	872	2
MNLI	Three-class NLI	4,000	9,815	3

SST-2 provides a clean binary classification task where the majority-class baseline is approximately 50.9%. Flat accuracy at this level across training epochs is an unambiguous failure signal.

MNLI provides a structurally different task requiring understanding of entailment relationships. The three-class structure creates the possibility of *negative transfer*: a model whose prediction distribution collapses to a single class will score below random chance on the non-majority class subpopulation. MNLI degradation rules out the “needs more training” explanation.

QNLI and QQP were planned but not evaluated. A GPU memory contention event (17.75 MiB free on an H100 NVL

with concurrent processes) caused an out-of-memory failure before the v9 training loop began.

4.3. Baselines

Zero-shot Mamba-130m: The pretrained base model evaluated on GLUE tasks using lm-evaluation-harness multiple-choice prompting, with no classification head and no fine-tuning. This establishes what the model achieves without any adaptation.

Random baseline: For SST-2 binary classification, random guessing yields $\sim 50\%$. For MNLI three-class classification, the theoretical random baseline is $\sim 33\%$.

4.4. Implementation Details

All experiments use the HuggingFace `state-spaces/mamba-130m-hf` checkpoint: a 24-layer causal Mamba-1 SSM with hidden dimension 768, inner dimension 1536, and the GPT-NeoX-20B tokenizer (50k vocabulary). Approximately 130M total parameters.

LoRA configuration: rank 8, $\alpha = 16$, dropout 0.05, targeting `in_proj`, `out_proj`, `x_proj` across all 24 layers. 1,484,288 trainable parameters (1.14%). The `dt_proj` layer was excluded from Condition A as the standard community configuration; an attempt to include it (v9) caused CUDA out-of-memory on the shared cluster before any training steps.

Compute: H100 NVL (93 GiB total; shared GPU environment). v8 completed successfully.

4.5. Evaluation Metrics

Per-epoch accuracy: Classification accuracy on the full validation split computed at the end of each epoch.

Per-batch cross-entropy loss: Logged at every gradient step. Primary aggregate of training optimization trajectory.

LoRA installation verification: Before accepting any result, we confirm (1) LoRA keys exist in the model state dictionary at expected paths, (2) LoRA weight matrices are non-zero after training, and (3) the trainable parameter count matches the theoretical expectation.

Gate criterion: SST-2 accuracy > 0.70 after three epochs.

5. Results

The central finding is unambiguous: projection-only LoRA on Mamba-130m does not produce classification learning.

Table 4. Zero-shot Mamba-130m baselines on GLUE.

Task	Zero-Shot Acc.	Random Baseline	Interpretation
SST-2	0.4908	~0.50	Near random; no classification prior
MNLI	0.3463	~0.33	Marginally above zero-shot baseline
QNLI	0.5056	~0.50	Near random
QQP	0.0000	~0.50	Systematic minority-class prediction

Table 6. MNLI accuracy across fine-tuning epochs (halted after epoch 2 due to monotonically worsening trend).

Evaluation Point	Accuracy	vs. Zero-Shot
Zero-shot baseline	0.3463	—
After Epoch 1	0.3326	−0.0137 (−1.4pp)
After Epoch 2	0.3234	−0.0229 (−2.3pp)

5.1. Zero-Shot GLUE Baselines

The SST-2 and MNLI zero-shot results confirm expected behavior for a causal language model with no classification instruction tuning. The QQP result (0.0000) is documented for reproducibility: Mamba-130m predicts the “not paraphrase” class for every QQP sample, most plausibly due to a tokenization or prompt-format incompatibility with the lm-evaluation-harness QQP template. It does not affect our primary conclusions.

5.2. Primary Result: SST-2 Fine-Tuning

Table 5. SST-2 accuracy across three fine-tuning epochs (MUST.WORK gate: > 0.70).

Evaluation Point	Accuracy	vs. Zero-Shot	vs. Gate
Zero-shot baseline	0.4908	—	−0.2092
After Epoch 1	0.5092	+0.0184	−0.1908
After Epoch 2	0.5092	+0.0184	−0.1908
After Epoch 3	0.5092	+0.0184	−0.1908
Gate result	—	—	FAIL

Three consecutive identical accuracy values of 0.5092 are the clearest possible indicator of zero learning. If any learning were occurring, the model’s predictions would shift from epoch to epoch, producing different accuracy values. Three identical values across three independent epochs, evaluated on 872 validation examples, indicate the model’s prediction distribution has not changed at all.

The net improvement over zero-shot is +1.84 percentage points. On $n = 872$ validation samples with $p \approx 0.5092$, the binomial standard error is $SE = \sqrt{p(1-p)/n} \approx 0.017$ (1.7pp). The observed gain is approximately 1 SE above the zero-shot baseline — within statistical noise. The 19.1 percentage-point gap between the observed 0.5092 and the gate criterion of 0.70 represents the gap between majority-class prediction and meaningful classification ability.

5.3. Secondary Result: MNLI Fine-Tuning and Negative Transfer

Both epochs show accuracy *below* the zero-shot baseline — negative transfer. This result rules out the “locally stable prediction regime” interpretation of the SST-2 result.

Something is moving: the model’s MNLI predictions are shifting with each epoch, but in the wrong direction. The gradient updates are coherent enough to steer the model but not coherent enough to move it toward correct classification labels.

The most plausible interpretation is classifier degeneracy: the classification head, updated without effective task-discriminative gradient, collapses predictions toward a single NLI class. The trajectory from 0.3463 toward 0.3234 is consistent with convergence toward the minority-class fraction.

The cross-task consistency — flat SST-2, degrading MNLI — strongly argues against any task-specific explanation.

5.4. LoRA Installation Verification

Table 7. LoRA installation verification checks (all PASS).

Check	Expected	C
lora_A keys in state dict	Present at all target paths	P
lora_B keys in state dict	Present at all target paths	P
Trainable parameter count	~1.14% of 130M	1
LoRA weights non-zero post-training	Non-zero (updated by gradient)	N

LoRA is installed correctly. The combination of Table 5 (zero classification learning) and Table 7 (correct LoRA installation) is the core finding: **API compatibility does not imply gradient compatibility.**

5.5. Loss Oscillation Pattern

Cross-entropy loss on SST-2 training batches oscillates between 0.65 and 0.73 across all three training epochs with no monotonic decrease trend. By comparison, a model successfully learning binary classification with cross-entropy loss and Adam typically shows consistent decrease from $\ln(2) \approx 0.693$ within the first epoch.

The oscillation range $[0.65, 0.73]$ centered near $\ln(2)$ is consistent with a model whose prediction distribution is not departing from near-50/50 confidence. The diagnostic signature of SSM gradient barrier failure is: loss oscillates in the range $[\ln(2) - \delta, \ln(2) + \delta]$ with no monotonic trend, while per-epoch accuracy remains flat or degrades.

5.6. Comparison to MambaPEFT Literature

Our SST-2 result (0.5092) contrasts with alxndrTL/mamba-peft (2024), which reports 90–92% SST-2 accuracy with LoRA $r=8$. The ~ 40 percentage-point gap cannot be attributed to statistical noise.

We do not interpret this as a contradiction. Our ranked hypothesis about the discrepancy source: the checkpoint type (base mamba-130m-hf vs. any instruction-tuned derivative) is our top candidate; the classification head design is second-most-likely; other training recipe details are lower-priority suspects. Our result establishes what a practitioner who applies the standard community LoRA configuration to the *base* Mamba checkpoint will observe.

6. Discussion

6.1. Key Findings and Interpretation

Our experiments reveal three jointly diagnostic patterns: SST-2 accuracy flat at 0.5092 across three epochs, MNLI accuracy degrading below zero-shot baseline, and training loss oscillating between 0.65 and 0.73 without convergence. Each pattern individually has multiple possible explanations; together, they converge on a single mechanistic interpretation.

The SSM scan acts as an effective gradient barrier for classification. The Mamba selective state space scan — implemented as a custom CUDA parallel associative scan (`selective_scan_cuda`) — sits between the classification head and the LoRA weight matrices in every one of Mamba-130m’s 24 layers. For the pretraining task (next-token prediction), this backward path through the scan is well-exercised. For a randomly-initialized classification head performing sequence-level prediction, the gradient signal is novel and apparently insufficient to drive meaningful weight updates in the LoRA matrices. The LoRA adapters receive gradient updates — their weights are non-zero and changed from initialization — but the updates carry no task-discriminative information.

Clarifying the gradient barrier: partial, not total. A critical nuance is that the barrier is not a complete block. The MNLI degradation result is evidence: a total barrier would produce flat MNLI accuracy (the classification head would also receive no gradient and remain locked). Instead, MNLI degrades monotonically — the head IS receiving gradient signal, and that signal is coherent enough to steer predictions toward a degenerate single-class regime. What the barrier blocks is not all gradient, but *task-discriminative* gradient. Non-discriminative gradient — noise, or gradient reflecting the model’s pretraining biases rather than the classification task’s label signal — passes backward through the scan. This is a medium-confidence mechanistic interpreta-

tion; directly logging per-layer gradient magnitudes would be required to confirm it.

This interpretation resolves the apparent tension: even though non-discriminative gradient reaches the LoRA weights and updates them, those updates are symmetrically distributed noise — they push predictions in different directions for different samples without any systematic class-directed bias — so their net effect on the population-level prediction distribution is zero, leaving validation accuracy unchanged despite the weights themselves changing.

This distinguishes two things the PEFT community frequently conflates: *PEFT API compatibility* and *PEFT gradient compatibility*. For transformer architectures, the attention mechanism is fully differentiable via standard PyTorch autograd, and gradient flows from any loss to any `nn.Linear` target reliably. For Mamba, the custom CUDA scan kernel interposes a computation whose backward pass does not effectively propagate classification-discriminative information to projection-layer adapters.

6.2. Limitations

Only SST-2 and MNLI evaluated. QNLI and QQP were planned but not completed due to GPU memory contention. The architectural gradient barrier mechanism is not task-specific, so the QNLI and QQP outcomes are expected to follow the same pattern.

Single seed. h-e1 used seed=42 only. The failure mode we observe — three consecutive identical accuracy values — is inconsistent with seed sensitivity. Architectural gradient barriers are structural properties of the computation graph, not seed-dependent.

No transformer control experiment. A GPT-2 or LLaMA model with an identical training setup would provide a direct comparison. We note that the MNLI active degradation provides indirect evidence that the training protocol is not globally broken: a completely broken optimizer would produce flat accuracy on MNLI as well, not monotonic degradation.

Gradient magnitudes not logged. The gradient barrier interpretation is supported by behavioral evidence but not by direct measurement of per-layer gradient magnitudes. Logging `grad.norm()` for the LoRA matrices at each training step would provide direct evidence.

MambaPEFT discrepancy unresolved. The 40 percentage-point gap between our SST-2 result and the community reference remains an open question.

6.3. Future Directions

SSM-kernel-level PEFT (`dt_proj`, B/C direct adaptation). The `dt_proj` layer controls the discretization of the SSM temporal dynamics — it is adjacent to the scan computation

rather than simply feeding into it. LoRA on `dt_proj` or direct adaptation of the B and C parameters would adapt parameters closer to the scan’s input, potentially bypassing the gradient barrier.

Prefix tuning for Mamba. Prefix tuning operates at the input embedding level, completely bypassing the gradient barrier by moving the adaptation point upstream of the scan. The engineering overhead is low and the method is architecture-agnostic.

Exact MambaPEFT replication. Before investing in new PEFT architectures, the most impactful step is to determine whether the MambaPEFT 90–92% result is reproducible. The ablation should follow the priority ordering: checkpoint type first, classification head design second.

6.4. Broader Impact

The SSM model family (Mamba, Falcon-Mamba, Jamba, Zamba2) is rapidly gaining adoption as a transformer alternative. Practitioners following community PEFT documentation will encounter the failure mode we document — LoRA installs, training runs, loss is non-zero — without a clear signal that no learning is occurring unless they measure per-epoch accuracy explicitly.

The zero-shot GLUE baselines for Mamba-130m (SST-2=0.4908, MNLI=0.3463, QNLI=0.5056, QQP=0.0000) are immediately reusable as reference values. The diagnostic signature (loss oscillation without trend, flat accuracy across epochs) is a practical early-stopping criterion. The broader conceptual contribution — that gradient-path analysis should be a first-class evaluation criterion for PEFT methods on non-transformer architectures — applies beyond Mamba.

7. Conclusion

We began with LoRA installed correctly on Mamba-130m — right target modules, right parameter counts, 1.14% trainable weights — and watched SST-2 accuracy sit at 50.9% for three consecutive training epochs, indistinguishable from the zero-shot baseline of 49.1%. We have now traced what that silence means.

The finding is not that LoRA fails in some general sense. The finding is that projection-only LoRA fails on Mamba for a specific and mechanistically coherent reason: the selective state space scan, implemented as a custom CUDA parallel associative scan, sits between the classification head and every LoRA weight matrix in every layer. For next-token prediction — the task this kernel was built for — gradient flows through it naturally. For a randomly-initialized classification head attached to the final hidden state, the gradient exists numerically but carries no task-discriminative

information to the projection layers. The loss oscillates. Accuracy does not move. And on MNLI, it moves in the wrong direction.

This is the distinction the field needs: **API compatibility is not gradient compatibility**. Mamba’s projection layers are `nn.Linear`. LoRA installs on them without complaint. And no learning occurs.

Our three contributions are grounded directly in this failure. First, we provide the empirical evidence: SST-2 accuracy of 0.5092 across three identical consecutive epochs on Mamba-130m, with MNLI degrading from 0.3463 to 0.3234 — both below the MUST_WORK gate and below any threshold for meaningful learning. Second, we document verified zero-shot GLUE baselines for `mamba-130m-hf` that are independently reusable by any future Mamba adaptation experiment. Third, we characterize the diagnostic signature — loss oscillation without monotonic decrease, combined with flat per-epoch accuracy — so practitioners can detect this failure mode during training.

Three directions follow directly from the failure analysis. Exact replication of the MambaPEFT result (90–92% SST-2) is the highest-priority step, following the ranked ablation order (checkpoint type first). If adaptation is needed at the scan level, `dt_proj` LoRA and direct B/C parameter adaptation operate inside or adjacent to the SSM computation — the theoretically motivated next step. And prefix tuning bypasses the scan entirely by adapting at the input embedding level.

The SSM case is not a dead end. Standard LoRA asks Mamba to learn through a wall. The contribution of this work is measuring the wall, describing its signature, and pointing to the door — architecture-aware PEFT methods that route through the SSM computation rather than around it.

References

- alxndrTL. mamba-peft: Parameter-efficient fine-tuning for mamba. <https://github.com/alxndrTL/mamba-peft>, 2024. Community repository; reports 90–92% SST-2 accuracy with LoRA rank 8 on Mamba-130m. Training recipe details not fully documented. **% UNVERIFIED: no formal publication; GitHub repository only.**
- Dao, T. and Gu, A. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint*, arXiv:2405.21060, 2024. URL <https://arxiv.org/abs/2405.21060>. Introduces Mamba-2 / SSD framework.
- Gao, L., Tow, J., Abbasi, B., Biderman, S., Black, S., DiPofi, A., Foster, C., Golding, L., Hsu, J., Le Noac’h,

- A., Li, H., McDonell, K., Muennighoff, N., Ociepa, C., Phang, J., Reynolds, L., Schoelkopf, H., Skowron, A., Sutawika, L., Tang, E., Thite, A., Wang, B., Wang, K., and Zou, A. A framework for few-shot language model evaluation. <https://github.com/EleutherAI/lm-evaluation-harness>, 2023. Version 0.4.x used in this work. % UNVERIFIED: version pinned to experiment environment; cite as software.
- Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint*, arXiv:2312.00752, 2023. URL <https://arxiv.org/abs/2312.00752>.
- Gu, A., Goel, K., and Ré, C. Efficiently modeling long sequences with structured state spaces. *arXiv preprint*, arXiv:2111.00396, 2022. URL <https://arxiv.org/abs/2111.00396>. Introduces S4; published at ICLR 2022.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. *arXiv preprint*, arXiv:2106.09685, 2022. URL <https://arxiv.org/abs/2106.09685>. Published at ICLR 2022.
- Li, X. L. and Liang, P. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of ACL-IJCNLP 2021*, 2021. URL <https://arxiv.org/abs/2101.00190>. arXiv:2101.00190.
- Liu, H., Tam, D., Muqeeth, M., Mohta, J., Huang, T., Bansal, M., and Raffel, C. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. *arXiv preprint*, arXiv:2205.05638, 2022. URL <https://arxiv.org/abs/2205.05638>. Introduces IA³; published at NeurIPS 2022.
- Liu, S.-Y., Wang, C.-Y., Yin, H., Molchanov, P., Wang, Y.-C. F., Cheng, K.-T., and Chen, M.-H. DoRA: Weight-decomposed low-rank adaptation. *arXiv preprint*, arXiv:2402.09353, 2024. URL <https://arxiv.org/abs/2402.09353>.
- Peng, B., Alcaide, E., Anthony, Q., Albalak, A., Arcadinho, S., Cao, H., Cheng, X., Chung, M., Grella, M., GV, K. K., He, X., Hou, H., Kazienko, P., Kocon, J., Kong, J., Koptyra, B., Lau, H., Mantri, K. S. I., Mom, F., Saito, A., Tang, X., Wang, B., Wind, J. S., Wozniak, S., Zhang, R., Zhang, Z., Zhao, Q., Zhou, P., Zhu, J., and Zhu, R.-J. RWKV: Reinventing RNNs for the transformer era. *arXiv preprint*, arXiv:2305.13048, 2023. URL <https://arxiv.org/abs/2305.13048>. Published at EMNLP 2023 Findings.
- Sinha, K., Pineau, J., Forde, J., Ke, R. N., and Larochelle, H. ML reproducibility challenge 2020. *arXiv preprint*, arXiv:2104.08691, 2021. URL <https://arxiv.org/abs/2104.08691>. % UNVERIFIED: arXiv ID from inferred source; confirm exact paper and authors before submission.
- Sun, Y., Dong, L., Huang, S., Ma, S., Xia, Y., Xue, J., Wang, J., and Wei, F. Retentive network: A successor to transformer for large language models. *arXiv preprint*, arXiv:2307.08621, 2023. URL <https://arxiv.org/abs/2307.08621>.
- Zhang, Q., Chen, M., Bukharin, A., Karampatziakis, N., He, P., Cheng, Y., Chen, W., and Zhao, T. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. *arXiv preprint*, arXiv:2303.10512, 2023. URL <https://arxiv.org/abs/2303.10512>. Published at ICLR 2023.